

# Day 13 - Practical Python: Project Structure, Packaging, Git Workflow

## Day 13 — Practical Python: Project Structure, Packaging, Git Workflow

### Objectives

- Learn the conventional way a real Python project's files and folders are organized, so an unfamiliar codebase at a job doesn't feel disorienting
- Understand `pyproject.toml` and the basics of managing a project's dependencies (the other packages it needs to run)
- Understand what all those extra files (`__pycache__`, `.gitignore`, and so on) that show up in real projects actually are, and why they're there
- Practice the real, everyday git workflow — not just `git init` once, but the actual back-and-forth of committing and branching as you genuinely work

### Why does file organization even matter?

So far, every day's exercises have lived in one single, simple folder, with just a `lesson.md`, `exercises.py`, and `solutions.py`. That's completely fine for learning exercises. But once a project grows to dozens or hundreds of files, worked on by more than one person, an organized, predictable structure becomes essential — so that anyone (including a brand-new teammate, or you yourself returning to a project six months later) can find what they're looking for by relying on well-known conventions, rather than having to explore an unfamiliar, ad-hoc mess from scratch every time.

### A conventional project layout

```
my_project/
├── .venv/                # virtual environment (Day 11) -- NEVER commit this to git
├── .gitignore
├── pyproject.toml        # project metadata + dependencies (explained below)
├── README.md
├── src/
│   ├── my_project/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   └── utils.py
│   └── tests/
│       ├── __init__.py
│       └── test_utils.py
```

This particular arrangement — putting your actual package inside a folder named `src/`, rather than directly at the very top level of the project — is commonly called the "**src layout**," and it's worth knowing both what it looks like and *why* experienced Python developers deliberately choose it. It might seem like unnecessary

extra nesting at first glance, but it forces you to properly `pip install` your own package (even in a special "editable" development mode, explained below) in order to import it at all — this catches an entire, surprisingly common category of bug ("it works, but only because I happened to be running it from exactly the right folder") before it ever has a chance to reach anyone else, including a new contributor to the project who might run things from a slightly different location.

## `.pyproject.toml` — the modern standard for describing a project and what it needs

```
[project]
name = "my_project"
version = "0.1.0"
description = "A short description"
requires-python = ">=3.10"
dependencies = [
    "requests>=2.28",
]

[project.optional-dependencies]
dev = ["pytest>=7.0"]

[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"
```

This single file (`.toml` is a plain-text configuration file format, not Python code itself) replaces what, in older Python projects you might still encounter, used to be spread awkwardly across several separate files (`setup.py`, `setup.cfg`, and a plain `requirements.txt`). You'll still see those older files in some existing, legacy codebases — recognize them when you see them, but write any brand-new project using `pyproject.toml`. The `dependencies` list names everything your code genuinely needs *in order to run at all*; `optional-dependencies.dev` separately lists things only needed while *developing* the project itself (like `pytest` for testing) — someone who just wants to *use* your finished package doesn't need testing tools pulled in alongside it for no reason.

Once you have a `pyproject.toml`, the standard way to install your own project while you're actively working on it is "editable mode" — this means any change you make to your source files takes effect immediately the next time you run your code, without needing to reinstall anything in between:

```
pip install -e .
```

## `.gitignore` — telling git which files it should never track

```
.venv/
__pycache__/
*.pyc
.pytest_cache/
*.egg-info/
.env
```

A `.gitignore` file tells git (the version control tool you'll actually use momentarily) which files and folders to simply ignore entirely — never track them, never suggest committing them. Here's what each of the above

actually is, and why it belongs on this list:

- `__pycache__` and `**.pyc` — when Python runs your code, it automatically compiles your human-readable `.py` source files into a lower-level, faster format called "bytecode," and caches (saves) that compiled version here so future runs can start slightly faster. It's entirely a *derived*, automatically-regenerated artifact — nobody hand-writes it, and it gets recreated automatically whenever needed. Committing it to git would be pure clutter, and could even occasionally cause confusing bugs for a teammate if a stale cached copy somehow lingered around.
- `.venv` — your virtual environment (Day 11) is specific to *your* particular computer, and can also be quite large. Your `pyproject.toml` (or `requirements.txt`) is the actual, portable source of truth that lets any teammate recreate their own separate, independent virtual environment on their own machine.
- `.env` — a common convention for a file holding secret values (passwords, private API keys, and similar) meant only for your own local setup. Accidentally committing genuine secrets to git is a real, well-documented category of security incident that happens to real companies — and critically, once something has been pushed to a shared git repository, you generally have to treat it as permanently compromised, even if you delete it again in a later commit, since git deliberately keeps a full history of everything that was ever committed.

## The real git workflow, applied to code you've actually written this week

You've been creating files all week without any version control at all. Here's the minimum realistic workflow, assuming you have git installed on your computer:

```
git init                                # run ONCE, at the very start of a project
git add .                               # stage every current file, getting it ready to be committed
git commit -m "Initial commit: week 1 exercises" # actually save that staged snapshot, with a message

# ... later, after you've changed some files ...
git status                             # see, in plain terms, what's changed since your last commit
git diff                               # see the EXACT lines that changed, added or removed
git add day12-testing-debugging/        # stage just this particular folder's changes
git commit -m "Add pytest tests for calculator module"

git log --oneline                       # see a compact, readable history of every past commit
```

**\*\*A commit message should explain *why* a change was made, not simply restate *what* changed\*\*** — `git diff` already shows you precisely *what* changed, in full detail; a message like `"Fix off-by-one error in complete_task"` will be far more useful to you six months from now than a vague `"update tasks.py"`, especially once you're looking back trying to understand a decision you no longer remember making.

**Branches** are used at essentially every real programming job, for anything beyond the smallest, most trivial one-line fix:

```
git checkout -b feature/add-remove-command # create AND switch to a brand-new branch, named for the feature
# ... make your changes, commit them normally on this branch ...
git push -u origin feature/add-remove-command # upload this branch to a shared, remote location
# from there, you'd open a "pull request" on GitHub/GitLab, invite a teammate to review it, and merge it back
```

A **branch** is essentially an independent, parallel line of work, separate from your project's main line of history — it lets you make and commit changes freely without affecting the main, shared version of the code until you (and, at a real job, typically at least one reviewing teammate) are confident it's ready. The core discipline worth adopting now, even while you're still learning solo: **avoid committing changes directly onto the `main` (or `master`) branch on any team project**. Working through a branch, plus a "pull request" for someone else to review before merging, both catches bugs earlier (a second pair of eyes, before the change reaches everyone else) and leaves a genuinely useful paper trail behind — months later, `git blame` (a command that shows you exactly which commit last changed any given line) can lead you straight back to the original pull request and its description, which is often exactly the context you need to safely understand or change that code later.

## Automated formatting and linting — an automated "style reviewer" for your code

Two tools you'll see wired into the setup of almost every real Python project's workflow:

- **black** — an intentionally opinionated auto-formatter. You simply run it, and it rewrites your code's formatting (spacing, line length, quote style, and more) to one single, consistent style across the entire project — permanently ending any team debate about exactly how code "should" be formatted, since the tool just decides for everyone.
- **ruff** (increasingly the modern standard, and a common, faster replacement for older tools like `flake8/pylint` that you might still encounter in some existing projects) — a fast **linter**, meaning it scans your code for likely bugs and bad patterns without actually running it: unused imports you forgot to remove, references to variables that were never defined, and other common mistake patterns.

```
pip install ruff black
ruff check .           # scan the whole project for problems
black .               # automatically reformat the whole project to one consistent style
```

At a real job, tools like these are usually run automatically as part of your **CI** ("Continuous Integration" — an automated system that runs checks on your code every time you push it), and often even earlier than that, via something called a "pre-commit hook" that runs automatically the moment you try to make a commit at all, before it's even allowed to complete. Getting comfortable running these tools yourself, locally, before you ever push your code, saves you the frustrating cycle of pushing code, waiting for CI, and then discovering it failed over a purely cosmetic formatting detail you could have caught yourself in seconds.

## Exercises

This is a hands-on, do-it-for-real day rather than an auto-graded one. Working directly inside this `day13-practical-python` folder:

- 1 Create a `src/greeter/` package containing an `__init__.py` (this file can be completely empty — its mere presence is what tells Python "treat this folder as an importable package") and a `core.py` file containing a function `greet(name: str) -> str` that returns `f"Hello, {name}!"`.
- 2 Write a `pyproject.toml` for this small package, naming it `greeter`, at version `0.1.0`.
- 3 Write `tests/test_core.py` containing a `pytest` test for `greet`.
- 4 Write a `.gitignore` covering `.venv/`, `__pycache__`, `*.pyc`, and `.pytest_cache/`.

- 5 If you have git installed: run `git init` right here in this folder, then make two genuinely separate commits — one for the package plus `pyproject.toml`, and a second, later one for the tests plus `.gitignore` — each with a commit message that explains *\*why\**, not merely *\*what\**.
- 6 Run `pip install -e .` from this folder, then confirm that `python -c "from greeter.core import greet; print(greet('Ana'))"` works correctly from anywhere on your computer (not just from inside this specific folder), as long as your virtual environment is still active.

A fully worked reference is provided in [solution/](#) if you get stuck on the packaging mechanics specifically — but make a genuine effort to run the actual git commands yourself; there's no substitute for typing them out and seeing the real output for yourself.